

Checklist

Designing Platform

1. Assess the 7 “C”s of Platform Quality

- ☐ **Cohesion:** Does the platform feel like one meaningful whole rather than a collection of unrelated tools?
- ☐ **Closure:** Within the platform’s chosen scope, are all necessary supporting pieces present or integrated?
- ☐ **Completeness:** Does the platform provide a complete user experience, including self-service, automation, debugging, documentation, training, and support where needed?
- ☐ **Consistency:** Are security, error handling, terminology, workflows, and other shared functions consistent across services?
- ☐ **Commensurate Value:** Can users gain useful value from adopting only part of the platform?
- ☐ **Connectedness:** Does the platform integrate effectively with surrounding systems such as identity, monitoring, and enterprise tooling?
- ☐ **Captivity:** Can users leave or replace the platform without redesigning their entire solution?
- ☐ Have we explicitly documented which of the 7 Cs matter most for our platform?
- ☐ Have we documented the trade-offs between them?
- ☐ Are we avoiding excessive **complexity** in the pursuit of maximizing every quality?

The source presents the 7 Cs as dimensions for evaluating design trade-offs rather than as a recipe in which every dimension must be maximized.

2. Build a “Fruit Salad,” Not Just a “Fruit Basket”

- ☐ Does the platform provide more value than simply installing or cataloging existing tools?
- ☐ Are individual platform components meaningfully integrated?

- ☐ Do users have shared account settings, identity, conventions, or workflows where appropriate?
- ☐ Are sensible defaults provided so users can get started quickly?
- ☐ Does automation eliminate repetitive work or manual integration?
- ☐ Can information flow between components—for example, linking operational metrics to deployments and source-code changes?
- ☐ Does the combined platform support useful scenarios that individual components cannot support as easily?
- ☐ Does the platform reduce user toil?
- ☐ Can users start small rather than having to adopt the entire platform before receiving value?
- ☐ Are platform constraints sufficiently opinionated to create a smooth experience without becoming unnecessarily restrictive?
- ☐ Are we measuring features by **user value**, rather than by how difficult they were to build?
- ☐ Are we resisting the temptation to hide every underlying tool behind a universal abstraction?

The text contrasts a “fruit basket” of loosely bundled tools with a “fruit salad” whose integration, defaults, automation, usability, and reduced toil create additional value.

3. Check the Horizontal and Vertical Architecture

- ☐ Have we identified the platform’s independent **vertical** components or services?
- ☐ Have we identified the shared **horizontal** capabilities that tie those components together?
- ☐ Does each horizontal capability have a clear reason to exist?
- ☐ Does it increase **reuse**?
- ☐ Does it support necessary **governance**?
- ☐ Does it improve **operations** across components?
- ☐ Have we clearly separated the data plane, control plane, and management/user-facing layers where relevant?
- ☐ Do users encounter unnecessary duplicate portals, consoles, or identities?
- ☐ Can independent platform components evolve without breaking the overall user experience?
- ☐ Where we want stronger horizontal standardization, is there a concrete user benefit rather than merely an architectural preference?

The “cantilevered platform” model treats platforms as vertical services joined by horizontal capabilities, with reuse, governance, and operational improvement given as major reasons for introducing those shared layers.

4. Decide Whether the Platform Will “Float” or “Sink”

- ☐ Have we explicitly defined what happens when the underlying cloud/base platform introduces functionality that duplicates ours?
- ☐ Do we periodically compare our platform capabilities with new capabilities from the base platform?
- ☐ Can redundant internal functionality be retired?
- ☐ Do we have a migration path for users when functionality is retired?
- ☐ Can the capacity freed by retirement be redirected toward higher-value capabilities?
- ☐ Are we avoiding sunk-cost reasoning when deciding whether to retain an internal feature?
- ☐ Are we evaluating the **opportunity cost** of maintaining duplicated functionality?
- ☐ Are platform components modular enough that individual capabilities can be replaced without changes cascading through the entire platform?
- ☐ Are we preventing the platform from continually gaining “weight” without shedding obsolete functionality?
- ☐ Are we prepared for the continuous evolution of the underlying platform?

A “floating” platform retires functionality that the base platform has commoditized and redirects effort toward new differentiated capabilities; keeping duplicated functionality can create maintenance and opportunity costs.

5. Avoid the “Grim Wrapper”

Before creating a new interface over an existing platform:

- ☐ Can our requirement be satisfied through **configuration** instead?
- ☐ Could we use **interception/hooks** to apply defaults or controls while retaining the original interface?
- ☐ Could **tracking and feedback** provide adequate governance without restricting the underlying API?
- ☐ Is a completely new wrapper genuinely necessary?
- ☐ If an operation is rejected, does the user receive a detailed explanation?

- ☐ Can our wrapper keep pace with changes and innovations in the underlying platform?
- ☐ Does the wrapper actually reduce cognitive load rather than merely reduce the number of API operations?
- ☐ Are users still able to benefit from external documentation, community knowledge, and skills?
- ☐ Are we avoiding a lowest-common-denominator abstraction across different technologies?
- ☐ Are important physical realities such as latency, cost, or service characteristics still visible where necessary?
- ☐ Have we accounted for the operational burden of running, securing, upgrading, and supporting our new abstraction?
- ☐ Have we considered whether the wrapper targets the control plane or runtime/data plane, and whether that distinction changes the trade-off?

The book warns that wrappers can lag behind rapidly evolving base platforms, create additional cognitive load, disconnect users from external expertise, conceal important characteristics, and introduce another component that must be operated.

6. Build Abstractions, Not Illusions

- ☐ Does each abstraction provide a **higher-level vocabulary** rather than merely rename underlying products?
- ☐ Are concepts named according to user intent or purpose instead of implementation details?
- ☐ Is the abstraction based on a real user or business domain?
- ☐ Does it form a coherent domain model rather than a collection of unrelated convenience APIs?
- ☐ Have we distinguished simple **composition** from genuine **abstraction**?
- ☐ Are the essential characteristics of the underlying system still represented?
- ☐ Have we avoided hiding inherent distributed-system concepts such as retries, backpressure, batching, timeouts, or ordering when users need to reason about them?
- ☐ Are important concepts represented explicitly rather than buried inside generic strings or numeric configuration fields?
- ☐ Does the abstraction make it easier to work with a complex problem, rather than pretending it's simple?
- ☐ Do we periodically reconsider what information is “essential,” recognizing that concerns such as cost, resource usage, and sustainability can change over time?

- ☐ If the interface looks extremely simple but requires extensive documentation to explain hidden behavior, have we reconsidered the model?

The text defines meaningful abstraction as a higher-level vocabulary based on domain understanding and warns that abstractions become “illusions” when they conceal essential complexity.

7. Design for Failure, Not Only the Happy Path

- ☐ Have we tested what happens when underlying components fail?
- ☐ Can users determine which underlying component caused a platform-level error?
- ☐ Do error messages preserve enough context to diagnose the root cause?
- ☐ Is there an equivalent of a **stack trace** linking high-level platform failures to their lower-level origin?
- ☐ Can experienced users or operators “open the hood” and inspect lower layers when necessary?
- ☐ Do we provide observability into the platform’s underlying components?
- ☐ Are failure modes documented as part of the abstraction?
- ☐ Does the platform provide users with a clear recovery path when something goes wrong?
- ☐ Have we considered partial failures rather than assuming an operation either succeeds completely or fails completely?
- ☐ Is the abstraction optimized for **debuggability** as well as ease of use?
- ☐ Is the learning curve gentle enough for normal users while still allowing specialists to investigate deeper problems?
- ☐ Do we have people who understand the underlying technology deeply enough to diagnose failures that cross abstraction boundaries?

The final chapter emphasizes that abstractions often break down during failure and recommends linking high-level errors back to their origin, designing abstractions for error scenarios, and retaining enough understanding of lower layers to diagnose unusual problems.

Final Go/No-Go Review

- ☐ The platform provides clear value beyond its individual components.
- ☐ The most important 7 Cs and their trade-offs are explicitly defined.
- ☐ Users can obtain value without adopting everything at once.
- ☐ Shared horizontal capabilities exist because they provide measurable value.

- ☐ The platform has a defined strategy for base-platform evolution.
- ☐ Redundant functionality can be removed without destabilizing the entire platform.
- ☐ Custom wrappers are used only where their benefits outweigh their long-term costs.
- ☐ Abstractions expose a meaningful domain rather than creating an illusion of simplicity.
- ☐ Failure scenarios are observable, diagnosable, and recoverable.
- ☐ The platform reduces cognitive load without hiding information users need to make correct decisions.
- ☐ The overall design remains simple enough to evolve over time.